

Polymorphismus

Aspekt	Beschreibung
Definition	Polymorphismus bedeutet, dass eine Methode oder ein Objekt in verschiedenen Kontexten unterschiedlich interpretiert oder verwendet werden kann. In Java wird dies durch Methodenüberladung (Overloading und Methodenüberschreibung (Overriding realisiert.
Ziel	Flexibilität und Erweiterbarkeit des Codes Ermöglichen von dynamischem Verhalten zur Laufzeit Förderung von Code-Wiederverwendung
Arten des Polymorphismus	1. Kompilierungszeit-Polymorphismus (Static Polymorphism): Wird zur Kompilierzeit entschieden, z. B. durch Methodenüberladung. 2. Laufzeit-Polymorphismus (Dynamic Polymorphism): Wird zur Laufzeit entschieden, z. B. durch Methodenüberschreibung und dynamische Bindung.
Methodenüberladung (Method Overloading)	Dieselbe Methode mit unterschiedlichen Parametern (Anzahl oder Typ der Argumente) innerhalb derselben Klasse. Wird zur Kompilierzeit entschieden. Beispiel: <code>void print(int a)</code> , <code>void print(double b)</code> .
Methodenüberschreibung (Method Overriding)	Eine Methode in einer Subklasse überschreibt eine Methode mit derselben Signatur in der Superklasse. Wird zur Laufzeit entschieden (dynamische Bindung).
Dynamische Bindung (Late Binding)	Bei einer Methode, die überschrieben wird, wird zur Laufzeit entschieden, welche Implementierung aufgerufen wird. Ermöglicht Polymorphismus durch Vererbung.
super vs. this bei Polymorphismus	<code>super.methodName()</code> ruft die Methode der Superklasse auf. <code>this.methodName()</code> ruft die Methode der aktuellen Klasse auf.
Polymorphismus mit Interfaces	Eine Klasse kann mehrere Interfaces implementieren, was Polymorphismus ohne Vererbung ermöglicht. Ermöglicht flexiblen und modularen Code.
Abstrakte Klassen und Polymorphismus	Eine abstrakte Klasse kann eine gemeinsame Schnittstelle für Subklassen definieren. Subklassen müssen die <code>abstract</code> -Methoden implementieren, wodurch einheitliche Schnittstellen entstehen.
Casting und Polymorphismus	Upcasting: Eine Subklasse wird als Superklasse behandelt (<code>Superklasse ref = new Subklasse();</code>). Downcasting: Eine Superklasse wird explizit in eine Subklasse umgewandelt (<code>Subklasse ref = (Subklasse) superRef;</code>).

Aspekt	Beschreibung
Virtuelle Methoden in Java	In Java sind Methoden standardmäßig virtuell, d. h., sie werden dynamisch zur Laufzeit aufgerufen (außer <code>static</code> , <code>private</code> oder <code>final</code> -Methoden).
Zusammenhang mit Vererbung	Vererbung ist die Grundlage für Polymorphismus. Eine Subklasse kann anstelle ihrer Superklasse verwendet werden.
Zusammenhang mit Kapselung	Polymorphismus arbeitet mit Kapselung, um Details der Implementierung zu verbergen und nur eine gemeinsame Schnittstelle bereitzustellen.
Zusammenhang mit Abstraktion	Polymorphismus ermöglicht Abstraktion, indem einheitliche Schnittstellen für unterschiedliche Implementierungen geschaffen werden.
Vorteile	Flexibilität und Erweiterbarkeit: Neue Klassen können leicht eingefügt werden, ohne bestehende Code-Basen zu ändern. Wiederverwendbarkeit: Gemeinsame Logik kann in Superklassen definiert werden. Verbesserte Wartbarkeit: Änderungen an einer Superklasse wirken sich auf alle Subklassen aus.
Nachteile	Erhöhter Speicherverbrauch: Dynamische Methodenbindung kann zusätzlichen Overhead verursachen. Schwierigere Fehlersuche: Es kann kompliziert sein, nachzuvollziehen, welche Methode zur Laufzeit tatsächlich aufgerufen wird.